

35.4. La fonction `sprite32()` — un grand personnage de 32×32

`#include <sprite32>` : `sprite32(numero, x, y, PERSO)` place un personnage de 32×32 — seize lutins en carré — d'un appel ; à côté, un 16×16 posé par `sprite16()`, pour comparer.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `sprite32(numero, x, y, perso)` place un personnage de 32×32 (un `Perso` de trente-deux rangées) : les lutins `numero` à `numero + 15`, en carré. C'est quatre `sprite16()` d'un coup, un par quart.

La ligne à écrire : `#include <sprite32>`.

Ses arguments : comme `sprite16()` ; le numéro du premier des seize lutins (**24 au plus** : la console n'en a que quarante). Un cinquième, `MIROIR_X`, le retourne.

Deux personnages, deux fichiers, à comparer : `perso_BONHOMME.cpp` (16 rangées : un 16×16) et `perso_GEANT.cpp` (32 rangées : un 32×32). Les deux sont des `Perso` — la liste `personnages.cpp` porte `#include <Perso>` UNE fois pour les deux — : c'est le nombre de rangées qui fait la taille. La source ne fait qu'appeler : `sprite16(..., BONHOMME)` pour l'un, `sprite32(..., GEANT)` pour l'autre.

À côté, pour comparer : `BONHOMME`, le personnage de 16×16 du 35.1, posé par `sprite16()`. Les deux sont des `Perso` : seize rangées pour l'un, trente-deux pour l'autre — c'est le nombre de rangées qui fait la taille, et la fonction qui va avec : `sprite16()` pour un 16×16 , `sprite32()` pour un 32×32 .

Attention à la place : seize lutins, c'est beaucoup. Avec un grand personnage, il en reste 24 pour tout le reste du jeu.

Essaie : `sprite32(0, 64, y, GEANT, MIROIR_X);` — GEANT regarde de l'autre côté.

On retrouve `sprite32` dans 3 leçons

[2.36](#), [2.37](#), [2.38](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1024 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          50 o ← la console, à l'allumage - la mise en route de la console
main()             535 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
RangerLutins        11 o ← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles        31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
```

```

Tuiles          320 o ← CopierTuiles  - 0 lettre sur 43, et 20 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert - la petite routine qui copie les lutins
Lettres gravées : aucune
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond,
EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
    sprite()   × 20   512 o sur place, dans main()
    image()    × 1    6 o sur place, dans main() + AttendreImage (28 o)
    sprite16() × 5    0 o sur place, dans main()
    sprite32() × 1    0 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- #include <sprite32> : un grand personnage de 32 × 32 ----
// Deux personnages, deux fichiers : perso_GEANT.cpp (32 × 32) et
// perso_BONHOMME.cpp (16 × 16), inclus par la liste personnages.cpp.
// Ici, UNE ligne verse la liste, et on ne fait que les APPELER :
// sprite32() pour le 32 × 32, sprite16() pour le 16 × 16.

#include <sprite32>          // sprite32() : seize lutins en carré
#include <sprite16>         // sprite16() : quatre lutins en carré
#include "personnages.cpp"  // la liste : GEANT et BONHOMME

```

2. UNE VARIABLE — Y

```

uint8_t y = 0; // la hauteur de GEANT, en pixels

```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    while (true) {
        image();
        if (y < 100) y = y + 1;          // GEANT descend, et s'arrête en 100
        sprite32(0, 64, y, GEANT);      // GEANT : les lutins 0 à 15
        sprite16(16, 16, 16, BONHOMME); // BONHOMME : les lutins 16 à 19, en haut à gauche
    }
}

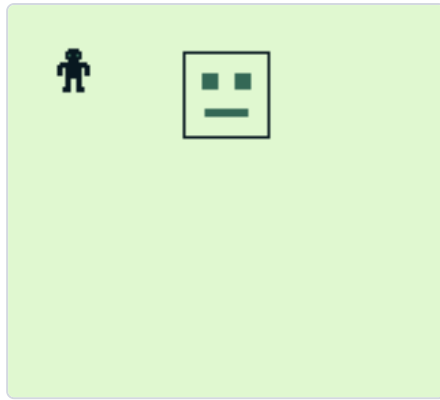
```

3. Ce qui se passe, étape par étape

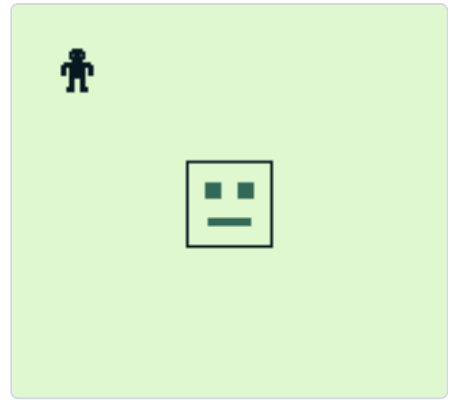
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



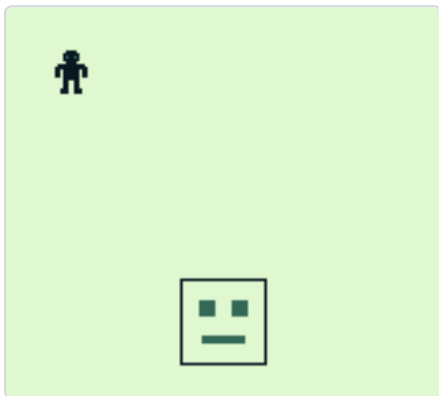
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Une grande tête de robot (GEANT) qui descend au milieu de l'écran et s'arrête en bas ; le petit bonhomme reste en haut à gauche.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ GEANT est arrivé en $y = 100$ (100)
- ✓ ses seize lutins font un carré de 32
- ✓ BONHOMME, en 16×16 , prend les lutins 16 à 19

Ensuite : **35.5. La fonction `cacher32()` — ôter un grand personnage** — chapitre 6 : Dessiner. · `gameboy3` — écrire une cartouche Game Boy en C++